YoursTech Academy

Full-Stack Web development

Start your journey to success with comprehensive courses and easy-to-follow learning methods!



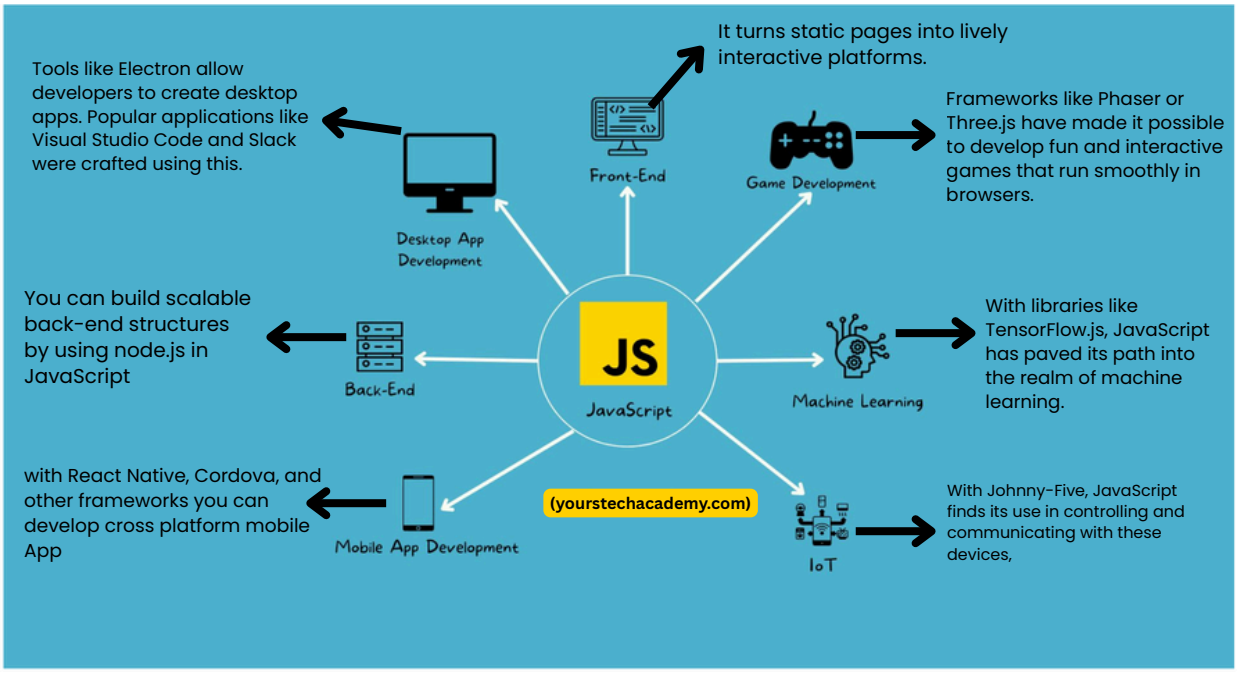
Presented by:
Bontu Salih

Section 14. Introduction to Javascript

What is Javascript?

JavaScript is a programming language used to create dynamic content for websites. It is a lightweight, cross-platform, and single-threaded programming language. It's an interpreted language that executes code line by line, providing more flexibility.
It is one of the three core technologies of the World Wide Web, working alongside HTML (which structures content) and CSS (which styles content).

What we can do with Javascript

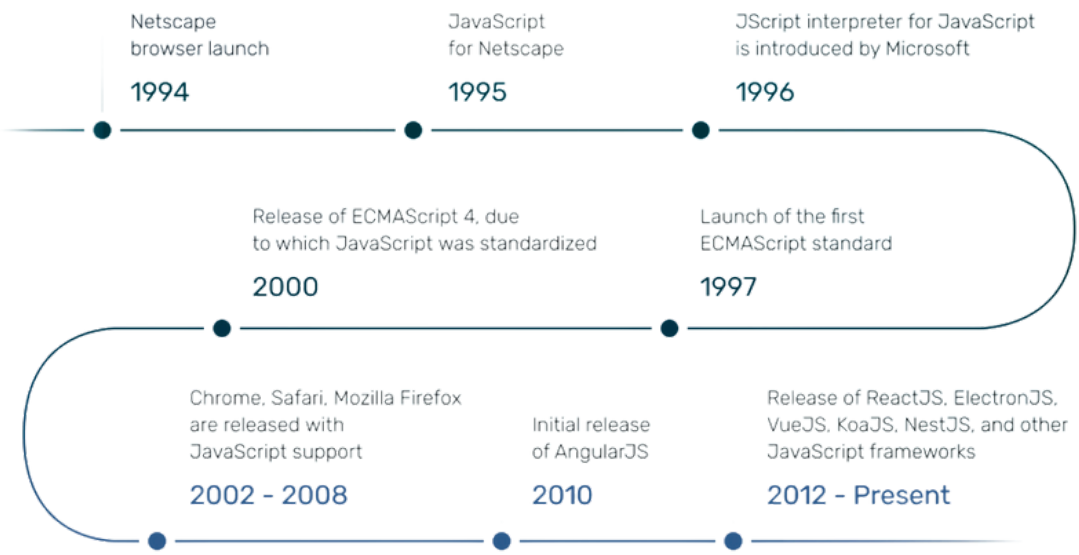


The birth of JavaScript (1993)

- JavaScript was created in **May 1993 by Brendan Eich**, a software engineer at **Netscape** Communications Corporation. The language was developed in a remarkably short timeframe of just **10 days.**

- Initially, it was named "**Mocha,**" then briefly renamed "**LiveScript.**" The final name, "**JavaScript,**" was chosen as a strategic marketing move to capitalize on the popularity of Java at the time, even though the two languages are fundamentally different.

- The primary purpose of JavaScript's creation was to add interactivity and dynamic features to web pages within Netscape's web browser, Netscape Navigator. This marked a significant step in the evolution of the web, moving beyond static HTML pages to more engaging and responsive user experiences.



JavaScript can be executed in several environments:

In a Web Browser:

Embedding in HTML: Create an HTML file and embed your JavaScript code within <script> tags.

```
<script>
    alert("This is a JavaScript alert!");
     console.log("This message appears in the browser console.")
</script>
```

Save this as an .html file (e.g., index.html) and open it with any web browser. The JavaScript will execute when the page loads.

Browser Developer Console: Open your browser's developer tools (usually by pressing F12 or right-clicking and selecting "Inspect"), navigate to the "Console" tab, and type or paste JavaScript code directly into it. Press Enter to execute.

Using Node.js (Server-Side or Standalone):

- Installation: Download and install Node.js from its official website.
- Running a File: Create a JavaScript file (e.g., my_script.js).

```
console.log("Hello from Node.js!");
```

Open your terminal or command prompt, navigate to the directory containing the file, and run it using the node command:

```
node my_script.js
```

How to Add JavaScript in HTML Document?

To add JavaScript in HTML document, several methods can be used. These methods include embedding **JavaScript** directly within the **HTML** file or linking an external JavaScript file.

Inline JavaScript

You can write JavaScript code directly inside the HTML element using the onclick, onmouseover, or other event handler attributes.

```
<html>
<head></head>
<body>
  <h2>
    Adding JavaScript in HTML Document
  </h2>
  <button onclick="alert('Button Clicked!')">
    Click Here
  </button>
</body>
</html>
```

Internal JavaScript (Within <script> Tag)

You can write JavaScript code inside the <script> tag within the HTML file. This is known as internal JavaScript and is commonly placed inside the <head> or <body> section of the HTML document.

1. JavaScript Code Inside <head> Tag

Placing JavaScript within the <head> section of an HTML document ensures that the script is loaded and executed as the page loads. This is useful for scripts that need to be initialized before the page content is rendered.

```
<head>
  <script>
    function myFun() {
      document.getElementById("demo")
        .innerHTML = "Content changed!";
    }
  </script>
</head>
```

2. JavaScript Code Inside <body> Tag

JavaScript can also be placed inside the <body> section of an HTML page. Typically, scripts placed at the end of the <body> load after the content, which can be useful if your script depends on the DOM being fully loaded.

```
<body>
<button type="button" onclick="myFun()">
      Click Here
   </button>
   <script>
      function myFun() {
         document.getElementById("demo")
            .innerHTML = "Content changed!";
      }
   </script>
</body>
```

External JavaScript (Using External File)

For larger projects or when reusing scripts across multiple HTML files, you can place your JavaScript code in an external .js file. This file is then linked to your HTML document using the src attribute within a <script> tag.

```
<head>
   <script src="script.js"></script>
</head>

<body>
   <script src="script.js"></script>
</body>
```

Section 15. Basic Javascript

Variables in JavaScript

a variable is a named storage location that holds a value. Think of a variable as a container for data, which can be referenced and manipulated throughout a computer program. The term "variable" indicates that the value it holds can change or "vary" during the execution of the program.

In JavaScript, variables are declared using the keywords **var, let, or const.**

1. var Keyword
The var keyword is used to declare a variable. It has a function-scoped or globally-scoped behaviour.

This is the original (older method) to declare variables in JavaScript. It is now discouraged in modern code due to its behavior.

```
var n = 5;
console.log(n);

var n = 20; // reassigning is allowed
console.log(n);
```

Output
5
20

2. let Keyword
The let keyword is introduced in ES6, has block scope and cannot be re-declared in the same scope.

```
let  n= 10;
n = 20; // Value can be updated

// let n = 15; //can not redeclare
console.log(n)
```

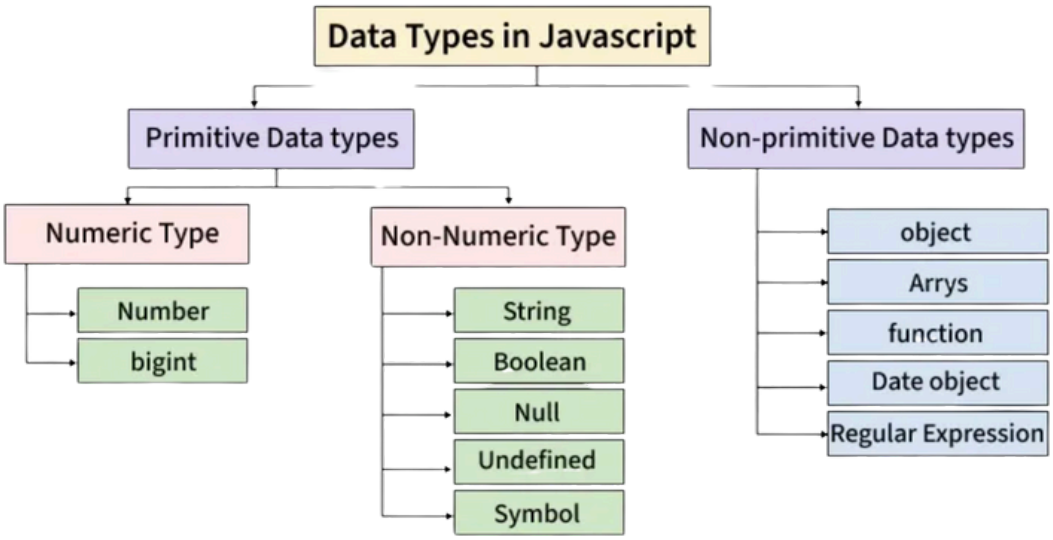
3. const Keyword
The const keyword declares variables that cannot be reassigned. It's block-scoped as well.

```
const n = 100;
// n = 200; This will throw an error
console.log(n)
```

Datatypes in JavaScript

In JavaScript, a data type is a **classification** that determines the kind of value a variable can hold. JavaScript is a **dynamically** typed language, meaning you do not have to explicitly define a variable's data type when it is declared. Instead, its type is determined at runtime based on the value it is assigned.

JavaScript's data types are categorized into two main groups: **primitive** and **non-primitive** (or reference).



Primitive data types

Primitive data types represent single, simple values. They are immutable, meaning they cannot be altered after creation.

String: Used for textual data. Strings are sequences of characters enclosed in single quotes ('...'), double quotes ("..."), or backticks (`...`).

```
let greeting = "Hello, World!";
```

Number: Represents numeric values, including **integers** and **floating-point** (decimal) numbers. It also includes special values like Infinity, -Infinity, and NaN (Not a Number).

```
let age = 30; // Integer
let price = 19.99; // Floating-point
```

Boolean: Represents a logical entity with only two possible values: true or false.

```
let isStudent = true;
```

Undefined: Indicates that a variable has been declared but has not been assigned a value.

```
let car; // The variable 'car' is undefined
```

Null: Represents the intentional absence of any object value. It is a special primitive value that is explicitly assigned.

```
let myCar = null;
```

Symbol: (Introduced in ES6) Represents a unique and immutable identifier. Symbols are primarily used to create unique keys for object properties to avoid name collisions.

```
const id = Symbol('unique id');
```

BigInt: (Introduced in ES2020) Represents integers with arbitrary precision. This is needed for numbers that exceed the safe integer limit of the Number type.

```
const bigNumber = 9007199254740991n;
```

Non-primitive (reference) data types

Unlike primitive types, non-primitive data types are mutable and can hold collections of data or more complex entities. When you assign a non-primitive value to a variable, the variable stores a reference to a memory location, not the value itself.

Object: The most fundamental non-primitive type. It is a collection of key-value pairs used to store complex data. Arrays and functions are technically a type of object.

```
let person = {
  name: "Alice",
  age: 25
};
```

Array: A special type of object for storing ordered collections of data.

```
let colors = ["red", "green", "blue"];
```

Function: Represents a block of reusable code that can be invoked. In JavaScript, functions are first-class objects, meaning they can be treated like any other variable.

How to check a data type

You can use the typeof operator to determine the data type of a variable.

```
let myVar = 10;
console.log(typeof myVar); // Output: "number"

myVar = "text";
console.log(typeof myVar); // Output: "string"

myVar = { name: "Bob" };
console.log(typeof myVar); // Output: "object"
```

Operators in Javascript

JavaScript operators are symbols or keywords used to perform operations on values and variables. They are the building blocks of JavaScript expressions and can manipulate data in various ways.

There are various operators supported by JavaScript:

1.Arithmetic Operators

Arithmetic Operators perform mathematical calculations like addition, subtraction, multiplication, etc.

const sum = 5 + 3	// Addition
const diff = 10 - 2	// Subtraction
const p = 4 * 2	// Multiplication
const q = 8 / 2	// Division
console.log(sum, diff, p, q)	

2.Assignment Operators

Assignment operators are used to assign values to variables. They can also perform operations like addition or multiplication while assigning the value.

```
let n = 10;
n += 5;
n *= 2;
console.log(n);
```

= assigns a value to a variable.
+= adds and assigns the result to the variable.
*= multiplies and assigns the result to the variable.

3.Comparison Operators

Comparison operators compare two values and return a boolean (true or false). They are useful for making decisions in conditional statements.

```
console.log(10 > 5);
console.log(10 === "10");
```

> checks if the left value is greater than the right.
=== checks for strict equality (both type and value).
Other operators include <, <=, >=, and !==.

4.Logical Operators

Logical operators are mainly used to perform the logical operations that determine the equality or difference between the values.

```
const a = true,   b = false;
console.log(a && b); // Logical AND
console.log(a || b); // Logical OR
```

&& returns true if both operands are true.
|| returns true if at least one operand is true.
! negates the boolean value.

Control Statements in JavaScript

A conditional statement in JavaScript executes a different block of code depending on whether a specified condition is true or false. They allow your programs to make decisions and provide different responses based on varying inputs or situations.

Types of conditional statements

if statement

This is the most basic type of conditional statement. It executes a block of code only if the condition inside the parentheses evaluates to true.

```
let age = 20;

if (age >= 18) {
  console.log("You are eligible to vote.");
}
```

In the example above, since the condition age >= 18 is true, the message is printed to the console.

if...else statement

This statement provides an alternative code path to be executed if the initial condition is false.

```
let age = 16;

if (age >= 18) {
  console.log("You are eligible to vote.");
} else {
  console.log("You are not old enough to vote.");
}
```

In this case, the if condition is false, so the code inside the else block runs instead.

if...else if...else statement

You can check multiple conditions in sequence using an else if block. JavaScript will execute the code block associated with the first true condition it finds and then exit the entire structure.

```
let temperature = 28;

if (temperature > 30) {
  console.log("It's a very hot day!");
} else if (temperature > 20) {
  console.log("It's a warm day.");
} else {
  console.log("It's a cool day.");
}
```

Here, the first condition is false, but the second condition (temperature > 20) is true, so "It's a warm day" is logged.

switch statement

A switch statement is an alternative to a long if...else if...else chain. It compares a single expression's value to a number of case values.

```
let day = 'Tuesday';

switch (day) {
  case 'Monday':
    console.log("Start of the work week.");
    break;
  case 'Tuesday':
    console.log("Second day of the week.");
    break;
  default:
    console.log("Just another day.");
}
```

The break keyword stops the switch statement from checking further cases. The default case is executed if none of the cases match.

Ternary operator

The ternary operator (? :) is a concise shorthand for a simple if...else statement. It takes a condition and returns one of two values depending on whether the condition is true or false.

```
let isRaining = true;
let activity = isRaining ? "Stay inside" : "Go outside";

console.log(activity); // Output: Stay inside
```

This single line of code achieves the same result as a more verbose if...else statement.

Functions in Javascript

Functions are reusable pieces of code that perform a specific task or calculate a value. Think of functions as a machine that takes some input, does some operations on it, and then produces an output. Here is an example of declaring a function:

```
function greet() {
  console.log("Hello, Bontu!");
}
```

In this example, we have declared a function called **greet**. Inside that function, we have a **console.log** that logs the message **Hello, Bontu!**. If we tried to run this code, we would not see the message appear in the console. This is because we need to call the function.

```
 function greet() {
   console.log("Hello, Bontu");
 }

 greet();      // "Hello, Bontu!"
```

We can make the greeting function reusable by using **parameters** and **arguments** instead of creating a new function each time.

Parameters are placeholders inside the function, while arguments are the actual values we pass when calling it.

```
function greet(name) {
  console.log("Hello, " + name + "!");
}

greet("Abdi"); // Hello, Abdi!
greet("Caltu"); // Hello, Caltu!
```

If you need your function to return a specific value, then you will need to use the **return** statement. Here is an example of using a return statement to **return** the sum of two values:

```
function calculateSum(num1, num2) {
  return num1 + num2;
}

console.log(calculateSum(3, 4)); // 7
```

Functions also support default **parameters,** allowing you to set default values for parameters. These default values are used if the function is called without an argument for that parameter. Here's an example:

```
function greetings(name = "Guest") {
  console.log("Hello, " + name + "!");
}

greetings(); // Hello, Guest!
greetings("Bontu"); // Hello, Bontu!
```

Another way to write functions in JavaScript is to create an arrow function expression. Here is how you can refactor the previous example to use arrow function syntax instead:

```
const greetings = (name) => {
  console.log("Hello, " + name + "!");
};
```

In this revised example, we are creating a const variable called greetings and assigning it an anonymous function. Most of the syntax will look familiar to you except for the missing function keyword and the addition of the arrow (=>) between the name parameter and the function body.

If your arrow function has no parameters, then you must use the parentheses like this:

```
const greetings = () => {
  console.log("Hello");
};
```

In summary, functions allow you to write reusable and organized code. They can take inputs (parameters), perform actions, and return outputs.

Arrays in JavaScript

An array in JavaScript is an ordered collection of values, each identified by a numeric index. The values in a JavaScript array can be of different data types, including numbers, strings, booleans, objects, and even other arrays.

To create an array in JavaScript, you can use square brackets, [], and separate the values with commas.
Here's an example:

```
let fruits = ["apple", "banana", "orange"];
```

In this example, we declare a variable fruits and assign it an array containing three string values: **apple, banana**, and **orange**.

One of the key characteristics of arrays is that they are **zero-indexed**, meaning that the first element in an array has an **index of 0,** the second element has an **index of 1**, and so on. You can access individual elements in an array using their index.
For example:

```
let fruits = ["apple", "banana", "orange"];
console.log(fruits[0]); // "apple"
console.log(fruits[2]); // "orange"
```

In this example, we use the index 0 to access the first element (apple) and the index 2 to access the third element (orange).

Arrays in JavaScript have a special **length** property that returns the number of elements in the array. You can access this property using the length keyword.
For example:

```
let fruits = ["apple", "banana", "orange"];
console.log(fruits.length); // 3
```

JavaScript arrays are **dynamic** you can easily add or remove items.
push() → adds elements to the end.
pop() → removes the last element.
unshift() → adds elements to the beginning.
shift() → removes the first element.

```
// Add to end
const fruits = ["apple", "banana"];
fruits.push("orange");
console.log(fruits); // ["apple", "banana", "orange"]

// Remove last
fruits.pop();
console.log(fruits); // ["apple", "banana"]

// Add to beginning
const numbers = [2, 3];
numbers.unshift(1);
console.log(numbers); // [1, 2, 3]

// Remove first
const colors = ["red", "green", "blue"];
colors.shift();
console.log(colors); // ["green", "blue"]
```

Object in JavaScript

An object in JavaScript is a data structure used to store **related information** as key value pairs.
Think of it like a box that holds labeled items — each label is a key (**property name**) and each item is its **value**.

```
const person = {
  name: "Alice",
  age: 30,
  city: "New York"
};
```

Here, person has three properties: name, age, and city.

Objects are very **important** in JavaScript — almost everything (arrays, functions, etc.) is an object or behaves like one.

Accessing Object Properties

You can access data inside an object in two main ways:

1. Dot Notation
This is the simplest and most common method.

```
console.log(person.name); // "Alice"
console.log(person.age);  // 30
```

Use this when the property name is a valid JavaScript identifier (no spaces, doesn't start with a number).

2. Bracket Notation

Used when property names contain spaces, numbers, or come from variables.

```
console.log(person["city"]);  // "New York"

const key = "age";
console.log(person[key]); // 30
```

Adding or Updating Properties

You can easily add new properties or update existing ones.

```
person.job = "Engineer";       // Add new property
person.age = 31;               // Update existing one

console.log(person);
```

Removing Properties

To removes a property from the original object.

```
delete person.job;
console.log(person.job); // undefined
```

Loops in JavaScript

A loop allows you to run a block of code multiple times automatically, instead of repeating it manually.
For example, if you want to print numbers from 1 to 50, you could write fifty console.log() statements
but with a loop, you can do it in just a few lines!

1. for Loop

Used when you know how many times you want the loop to run.

```
for (let i = 1; i <= 5; i++) {
  console.log("Number:", i);
}
```

Explanation:
let i = 1 → start value
i <= 5 → stop condition
i++ → increase by 1 after each loop

Break and Continue
break → stops the loop completely.
continue → skips the current iteration and moves to the next.

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) continue; // skip 3
  if (i === 5) break;    // stop at 5
  console.log(i);
}
```

2. while Loop

Used when you don't know exactly how many times the code should repeat

The while loop is meant for situations where the number of iterations isn't fixed in advance
we keep looping as long as a condition remains **true**.

```
let password;
while (password !== "1234") {
  password = prompt("Enter your password:");
}
```

Here, you don't know how many times the user will enter the wrong password it depends on their input.

3. do...while Loop

The do...while loop is similar to the while loop, but it guarantees that the code block will execute at least once before checking the condition.

```
let x = 0;
do {
  console.log("Value of x:", x);
  x++;
} while (x < 2);
```

4. for...of loop

The for...of loop provides a concise way to iterate over the values of iterable objects.

```
const fruits = ["apple", "banana", "cherry"];

for (const fruit of fruits) {
  console.log("I like", fruit);
}
```

This loop iterates through each element in the fruits array and prints it.

5. forEach method (for arrays)

The forEach method is a higher-order function available on arrays that executes a provided function once for each array element.

```
const numbers = [10, 20, 30];

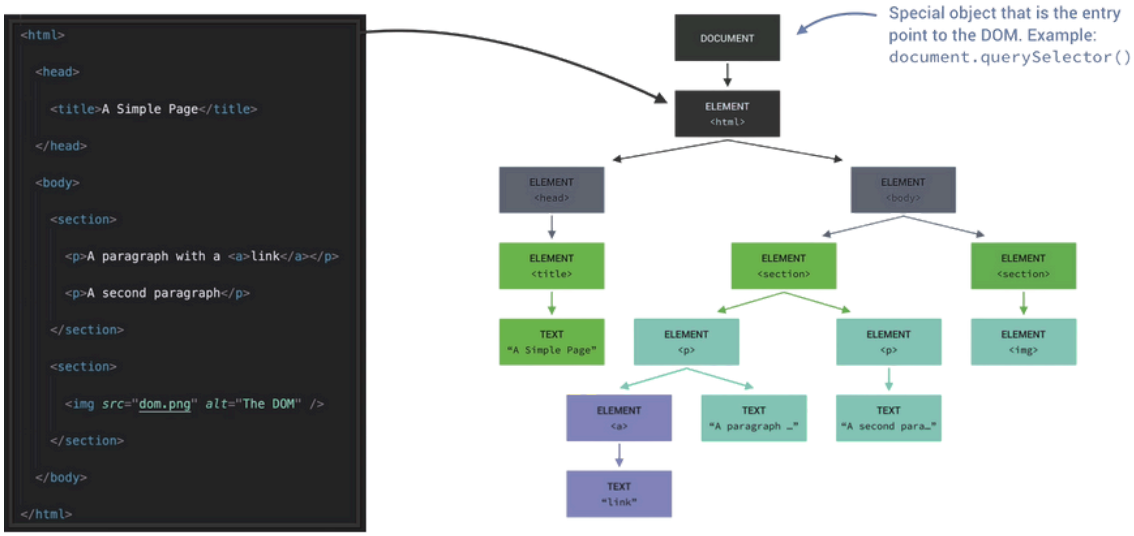
numbers.forEach(function(number) {
  console.log("Number is:", number);
});
```

Section 16. Document Object Model (DOM) in JavaScript

DOM is a programming interface that allows us to create, change, or remove elements from the document. We can also add events to these elements to make our page more dynamic.

The DOM views an HTML document as a tree of nodes. A node represents an HTML element.

Let's take a look at this HTML code to better understand the DOM tree structure.



We can access these elements in the document and make changes to them using JavaScript.

Selecting elements: Using methods like getElementById(), getElementsByClassName(), getElementsByTagName(), and querySelector() or querySelectorAll().

Modifying content: Changing text content with textContent or HTML content with innerHTML.

Changing attributes: Setting, getting, or removing attributes using setAttribute(), getAttribute(), and removeAttribute().

Styling: Modifying CSS properties using the style object of an element.

Creating and removing elements: Dynamically adding new elements with createElement() and appendChild(), or removing existing ones with removeChild().

Event handling: Attaching event listeners (e.g., click, mouseover) to elements using addEventListener() to make pages interactive.

Section 17. Asynchronous JavaScript

Asynchronous JavaScript is a programming approach that enables the non-blocking execution of tasks, allowing concurrent operations, improved responsiveness, and efficient handling of time-consuming operations in web applications, JavaScript is a **single-threaded** and synchronous language. The code is executed in order one at a time, But Javascript may appear to be asynchronous in some situations.

There are several methods that can be used to perform asynchronous javascript tasks, which are listed below:

```
function processOrderSync() {
  console.log("Order received");
  console.log("Processing payment...");
  console.log("Order completed");
}

processOrderSync();
console.log("Customer can browse now");
```

```
function processOrderAsync() {
  console.log("Order received");

  setTimeout(() => {
    console.log("Processing payment...");
  }, 2000);

  setTimeout(() => {
    console.log("Order completed");
  }, 4000);
}

processOrderAsync();
console.log("Customer can browse now");
```

1: Using callback

Callbacks are functions passed as arguments to be executed after an asynchronous operation completes. They are used in asynchronous JavaScript to handle responses and ensure non-blocking execution

```
function myFunction(callback) {
    setTimeout(() => {
        const data = { name: "Bontu Salih", age: 26 };
        callback(data);
    }, 3000);
}

myFunction((data) => {
    console.log("Data:", data);
});
```

2: Using Promises

Promises are objects representing the eventual completion (or failure) of an asynchronous operation, providing better handling of asynchronous code with .then() and .catch().

Promise Has three State

- Pending
- Fulfilled
- Rejected

```
function mydata() {

   return new Promise((resolve, reject) => {
      setTimeout(() => {
         const data = { name: "Bontu Salih", age: 26 };
         resolve(data);
      }, 2000);
   });
}

mydata()
   .then((data) => {
      console.log("Data:", data);
   })
   .catch((error) => {
      console.error("Error:", error);
   });
```

3. Async and Await

Async and Await in JavaScript are used to simplify handling asynchronous operations using promises. By enabling asynchronous code to appear synchronous, they enhance code readability and make it easier to manage complex asynchronous flows.

```
async function fetchData() {
  const response = await fetch("https://jsonplaceholder.typicode.com/posts/1");
  const data = await response.json();
  console.log(data);
}

fetchData();
```

Async Function

The async function allows us to write promise-based code as if it were synchronous. This ensures that the execution thread is not blocked. Async functions always return a promise. If a value is returned that is not a promise, JavaScript automatically wraps it in a resolved promise.

```
const getData = async () => {
   let data = "Hello World";
   return data;
}

getData().then(data => console.log(data));
```

Await Keyword

The await keyword is used to wait for a promise to resolve. It can only be used within an async block. Await makes the code wait until the promise returns a result, allowing for cleaner and more manageable asynchronous code.

```
const getData = async () => {
   let y = await "Hello World";
   console.log(y);
}

console.log(1);
getData();
console.log(2);
```

Error Handling in Async/Await

JavaScript provides predefined arguments for handling promises: resolve and reject.

- resolve: Used when an asynchronous task is completed successfully.
- reject: Used when an asynchronous task fails, providing the reason for failure.

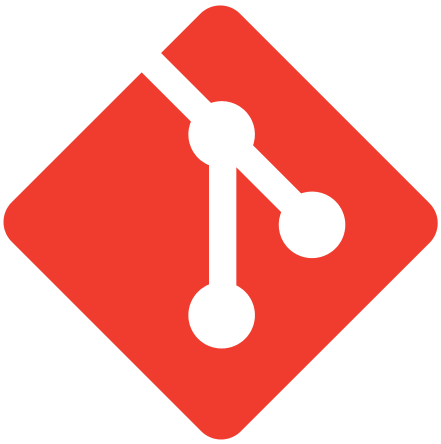
```
async function fetchData() {
 try {
   let response = await fetch('https://api.example.com/data');
   let data = await response.json();
   console.log(data);
 } catch (error) {
   console.error('Error fetching data:', error);
 }
}
```

Section 18. Version Constrol GIT and GitHub

What is GIT and GitHub

Git is a free and open-source distributed version control system (DVCS) designed to track changes in files, primarily source code during software development. It enables individuals and teams to manage different versions of a project's files, collaborate effectively, and revert to previous states if needed.

GitHub is a web-based platform used for hosting Git repositories and facilitating collaborative software development. It is built around Git, the open-source distributed version control system (VCS), but expands upon it with a user-friendly interface and numerous integrated collaboration and project management tools.





In Short Terms:

Git → program on your computer
GitHub → website where Git projects live

How to Install Git

On Windows
Go to Google → search: "Git for Windows"
Open the official site (git-scm.com)
Download and install:
Click Next on most options (default is fine)
After install, open:
Command Prompt or Git Bash

Check That Git Installed

Open terminal / command prompt and type
git --version

You should see something like:

git version 2.xx.x

On macOS
You have 2 easy ways:
Option 1 — Using Homebrew (recommended)
If you already have Homebrew:
brew install git
Option 2 — Xcode tools
Just run this in Terminal:
git --version
If Git isn't installed, it will ask: "install developer tools?" → choose Yes.

Most Important Basic GIT Commands

Set Your Git Identity: Git needs your name and email for commits.

git config --global user.name "Your Name"
git config --global user.email "youremail@example.com"

You can check them with:
git config --global --list

FOLDER & FILE COMMANDS (Before Git)

- mkdir project-name # create folder
- cd project-name # enter folder
- touch index.html # create file
- ls # list files
- ls -a # list including hidden (.git)
- rm file.js # delete file
- rm -r folder-name # delete folder

START GIT IN A PROJECT

- git init # initialize git repository
- git status # check project status

ADD & COMMIT (SAVE WORK)

- git add file.js # add specific file
- git add . # add all files
- git commit -m "message" # commit changes

VIEW HISTORY & CHANGES

- git log # see commit history
- git log --oneline # short history
- git diff # see unstaged changes
- git diff --staged # see staged changes

PUSH & PULL (SYNC WITH GITHUB)

- git branch -M main # rename branch to main
- git push -u origin main # first push to GitHub
- git push # push next commits
- git pull origin main # pull latest changes

CLONE EXISTING PROJECT

- git clone URL # copy repo from GitHub

UNDO & FIX MISTAKES (IMPORTANT)

- git restore file.js # undo file changes
- git restore --staged file.js # unstage file
- git reset HEAD~1 # undo last commit (keep files)
- git reset --hard HEAD~1 # delete last commit (danger!)

STASH (SAVE WORK WITHOUT COMMIT)STASH (SAVE WORK WITHOUT COMMIT)

- git stash # save work temporarily
- git stash list # view stashes
- git stash pop # restore latest stash

COMMON ERRORS & FIXES

- git remote remove origin # remove remote
- git fetch # fetch without merge
- git pull --rebase # clean pull